Perusall Exercise

Summary

Thank you for your interest in the Sr. Software Engineer role at Perusall.

As the next step in our interview process, we'd like you to complete a small full-stack development exercise. The exercise should not take more than 4 to 5 hours.

Our goal is to evaluate your technical skills, code quality, design decisions, and problem-solving approach.

This exercise is strictly for candidate evaluation. Perusall will not use your submitted code or work in its products or commercial operations.

Exercise Details

The exercise calls for building a simple Collaborative Reading App.

Build a simple application where college instructors can assign readings and college students can discuss them through annotations.

What is an annotation?

An annotation is a comment a student makes about assigned reading material. For example, while reading an article about machine learning:

- **Sarah:** "What's the difference between supervised and unsupervised learning?"
- **Michael:** "Supervised learning uses labeled data, while unsupervised learning identifies patterns."

Annotations are typically associated with specific passages. For this exercise, an annotation is simply a comment associated with an entire reading.

What is a reading?

A reading is educational material assigned by an instructor for students to study and discuss.

Examples include:

- A chapter from a textbook
- A research paper
- A PDF document
- An academic article
- An assigned passage of text

For this exercise a reading is simply an item with a title and description.

Requirements

1. Users & Roles

- Support two roles: Instructor and Student.
- Allow users to select their identity to simulate logging in.
- Display appropriate functionality based on the selected role.

2. Reading Management

- Instructors can create readings with a title and description.
- Students can browse and select readings.
- Display the total number of annotations for each reading.
- Persist readings in the database.

3. Annotations

- Students can add comments to readings.

- Display each annotation with the author's name and timestamp.
- Show annotations in reverse chronological order.
- Display new annotations without refreshing the page.
- Prevent empty annotations and persist all submissions.

4. Instructor Overview

- Show total annotations per reading.
- Show the number of unique students participating.
- Allow instructors to review all annotations for a reading.

5. User Experience

- Provide an intuitive interface for navigating readings and discussions.
- Display helpful success and error messages.
- Handle empty states gracefully.

We are especially interested in seeing how you imagine the interface. Feel free to use any styling or component libraries you like.

Optional Enhancements

If time permits, consider adding:

- **Filtering:** Filter annotations by student.
- **Editing:** Allow students to edit or delete their annotations.
- **Replies:** Allow students to reply to other annotations.
- **Dashboard:** Display student participation across readings.
- **Search:** Search readings by title.
- **Testing:** Add automated tests.

These enhancements are entirely optional.

Expectations

You do **not** need to:

- Implement production-grade authentication or authorization
- Implement production-grade security or infrastructure
- Deploy the application to the cloud
- Configure CI/CD
- Handle enterprise-scale data
- Recreate Perusall's annotation interface
- Build document rendering or PDF annotation functionality

You **should**:

- Write code that works and runs locally
- Keep the implementation understandable and reasonably organized
- Include a short `README.md` with local setup instructions
- Make intentional choices around architecture, data modeling, and UI
- Be prepared to explain the decisions you made
- Be prepared to walk us through your code during the next interview

AI & Vibe Coding

Vibe coding is absolutely welcome. You are encouraged to use AI-assisted development tools such as Cursor, Claude Code, ChatGPT, GitHub Copilot, or similar tools.

We care about your ability to deliver working software, make sound engineering decisions, and solve problems—not whether you manually typed every line of code.

However, you must understand your implementation and be prepared to explain, debug, or modify any part of it during the interview.

Submission

1. Submit a public GitHub or Bitbucket repository.
2. Include a `README.md` with setup and execution instructions.
3. Include sample data or instructions for populating the database.
4. Send your repository link to Bobby.

Be prepared to walk through your solution and discuss your technical decisions during a panel interview with the Perusall technical team.

We look forward to seeing your app!